POV 1 — From fixed test suites to evidence-gap-guided AI assurance

Status: Internal point of view, prepared for review in this repository; not externally published or independently peer-reviewed. **Audience:** AI assurance, product-security, evaluation, and governance teams. **Question:** Can an AI assurance workflow reduce redundant testing without confusing reduced test volume with stronger assurance?

Executive position

AI assurance should be treated as an evidence-management problem, not a checklist-completion problem. A fixed suite is repeatable and easy to govern, but it may spend effort repeating well-covered checks while leaving high-risk areas unexamined. We propose an evidence-gap-guided assurance (EGAA) loop: score the current evidence by confidence, freshness, and evidence mode; estimate risk-weighted gaps; then choose the next eligible test by expected gap reduction per cost, subject to explicit dependency and execution budgets.

The approach is a prioritization aid, not a substitute for a minimum required test suite, expert review, or a safety case. Fewer tests do not imply lower risk. A system with missing or weak evidence must remain explicitly incomplete.

Why this matters

Generative AI risk changes with the model, prompt, tools, data, and deployment context. A report that counts executed tests alone can obscure what those tests actually establish. NIST's AI RMF is intended to help organizations incorporate trustworthiness into AI design, development, use, and evaluation; its Generative AI Profile describes risk-management actions for generative-AI-specific risks [1, 2]. AAOS translates that general risk-management intent into an inspectable prototype evidence loop.

Proposed approach

1. Define an explicit risk taxonomy and weight dimensions according to impact.
2. Record each result with provenance: risk, test, outcome, execution status, provider, configuration, source, timestamp, confidence, strength, and evidence mode.
3. Discount evidence that is old, uncertain, mocked, or simulated. Failed, skipped, timed-out, and unknown results do not earn coverage.
4. Increase uncertainty when comparable evidence contradicts.
5. Estimate the evidence gap for each risk and score each eligible test by expected risk-weighted gap reduction divided by its estimated cost.
6. Select deterministically, record alternatives and exclusion reasons, execute within a test/time budget, and recompute gaps.
7. Stop with an explicit reason and label unresolved material gaps. Never turn a stopped run into a pass or certification.

The method supports a baseline suite required by policy. Adaptivity should allocate additional tests, not waive mandatory gates without an independently approved policy.

What AAOS implements

The prototype has eight risk dimensions and an evidence ledger. Evidence value uses:

`strength × confidence × freshness × evidence-mode factor`

Freshness decays with a 30-day half-life; the current mode factors are `real = 1.0`, `mocked = 0.55`, and `simulated = 0.35`. Risk gaps incorporate coverage and uncertainty/contradiction. The deterministic selector prioritizes risk-weighted expected gap reduction per estimated cost and records its considered candidates. The API persists evidence and selection decisions. Adaptive execution is restricted to mock provider mode.

The built-in security executor is only three static probes with keyword-oriented analysis. Several catalog tests are deliberately unavailable until their dependency exists. EGAA currently cannot interrupt a blocking Python callback; subprocess adapters have separate timeouts. These boundaries are part of the evidence record and deployment guidance, not hidden implementation details.

Prototype benchmark and interpretation

The included deterministic benchmark is synthetic and is not model-safety evidence:

Fixture scenario	Fixed tests	Adaptive tests	Fixed weighted coverage	Adaptive weighted coverage	Interpretation
Strong existing evidence	6	2	0.7605	0.7605	Same fixture coverage with fewer selected checks
Seeded security gap	6	1	0.4004	0.3752	Both detect the seeded prompt-injection failure; the adaptive loop stops immediately on the critical finding and reports lower coverage
Contradictory provider evidence	6	2	1.0000	1.0000	Same fixture coverage with fewer selected checks

These are three hand-authored, deterministic scenarios from `GET /assurance/benchmark`, not a statistically representative evaluation. The benchmark stops the seeded-security scenario after the critical finding. Costs and evidence strengths are configured assumptions. The result supports a hypothesis about test prioritization only; it does not establish improved real-world safety, recall, or cost-effectiveness.

Recommendations

- **?** Keep fixed, policy-required minimum controls separate from adaptive additions.
- **?** Validate the risk taxonomy, weights, half-life, confidence, mode factors, and cost model with domain owners before operational use.
- **?** Benchmark against randomized and risk-prioritized fixed baselines using real, authorized, independently labeled cases.
- **?** Report selection rationale, untested risks, failures, and provenance alongside every score.
- **?** Test stability under changed ordering, duplicated evidence, stale data, and adversarial inputs.
- **?** Require human sign-off before claims about safety or release readiness.

Conclusion

Evidence-gap guidance can make assurance work more targeted and more explainable, but only when uncertainty and failures remain visible. AAOS is an early prototype of this workflow. Its current benchmark is useful for debugging the algorithm, not for making deployment decisions.

References

1. NIST, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, 2023.
2. C. Autio et al., Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1, 2024.
3. AAOS implementation: EGAA engine [../backend/app/engines/assurance/egaa.py], synthetic benchmark [../backend/app/engines/assurance/benchmark.py], and limitations [KNOWN_LIMITATIONS.md].